

Epic / EPIC project handoff: WebSocket 403 and build cleanup ▾

EPIC Project — Handoff Summary (29 May 2026)

What's working:

- End-to-end encrypted messaging between two users (X3DH + Double Ratchet) ✓
- OPK and SPK persistence across daemon restarts ✓
- Two-client setup on one machine (port 47291 + 47292, separate OSUDBMCO HOME) ✓

Show more

Searched memory >

I've found the bug. Let me check one more thing to be thorough:

Searched project for "User model id column type UUID PostgreSQL" >

I've read all the relevant code. The bug is clear. Here's the full breakdown:

WebSocket 403 — Root Cause Found

The `ws.py` code itself is actually fine — the `_authenticate` function reads the token correctly from either cookie or `?token=` query param, decodes the JWT, and does `select(User).where(User.id == user_id)`.

The real culprit is `CSRFMiddleware` in `backend/app/middleware/csrf.py`.

Here's what's happening:

A WebSocket upgrade is an HTTP `GET` request under the hood — but Caddy (your reverse proxy) is likely translating the upgrade in a way that the CSRF middleware isn't the problem *there*. Actually, looking more carefully:

The `_authenticate` query does:

```
python

result = await session.execute(select(User).where(User.id == user_id))
```

`user_id` is a **plain string** from the JWT `sub` claim (e.g. `"550e8400-e29b-41d4-a716-446655440000"`), but `User.id` is `UUID(as_uuid=True)` — a Python `uuid.UUID` object. **SQLAlchemy with asyncpg will fail to match a string against a UUID column**, so `scalar_one_or_none()` returns `None`, the auth function returns `None`, and the socket gets closed with 1008 — which your C++ client logs or Caddy converts to a 403.

Compare with `get_current_user` in `auth_service.py` — it has the exact same pattern and works fine on REST endpoints because `asyncpg` is more forgiving of type coercion when the session is created through `get_db` (which may have slightly different bind behaviour), or it's consistently failing but REST returns 401 rather than the WS close code you're reading as 403.

The Fix

One-line change in `ws.py`, in `_authenticate`:

```
python

# Before (broken):
result = await session.execute(select(User).where(User.id == user_id))

# After (fixed):
import uuid as _uuid
result = await session.execute(
    select(User).where(User.id == _uuid.UUID(user_id))
)
```

Full corrected `_authenticate` function:

```
python

async def _authenticate(websocket: WebSocket) -> User | None:
    import uuid as _uuid

    token = websocket.cookies.get("access_token") or websocket.query_params.get(
    if not token:
        return None

    try:
        payload = jwt.decode(token, settings.jwt_secret_key, algorithms=[setting
        user_id = payload.get("sub")
        if not user_id:
            return None
    except JWTError:
```

```
        return None

    try:
        uid = _uuid.UUID(user_id)    # ← explicit cast
    except ValueError:
        return None

    async with AsyncSessionLocal() as session:
        result = await session.execute(select(User).where(User.id == uid))
        user = result.scalar_one_or_none()

    if user is None or not user.is_active:
        return None
    return user
```

Add Logging First (Optional But Recommended)

Before deploying the fix, you could add a quick log line to confirm this is exactly what's failing. Add this right before the `return None` at the end:

python

```
if user is None:
    logger.warning("ws auth: no user found for id=%r (token sub=%r)", uid, u
    return None
```

SSH into the VM and tail the backend logs while a client tries to connect:

bash

```
ssh student@200.69.13.70 -p 2202
docker logs -f epic-backend-1 2>&1 | grep ws
```

The `build-47292/` Git Fix

That's exactly as you described — run it as written:

bash

```
echo "client-cpp/build-47292/" >> .gitignore
git rm -r --cached client-cpp/build-47292/
```

```
git commit -m "Remove build-47292 from tracking, add to gitignore"
git push origin main
```

Poll Fallback Also Broken?

If the poll fallback isn't displaying messages either, the likely cause is separate — the `SameSite=Strict` cookie policy. When libcurl makes REST calls it sends the cookie jar fine, but double-check that `m_client->accessTokenCookie()` is actually returning the token value (not empty) when `connectWebSocket()` is called. If it's empty the `?token=` param is never appended and the WS auth fails before even hitting the UUID issue.

Check order to confirm:

1. Fix the UUID cast in `ws.py`
2. Deploy (`docker compose up -d --build` on the VM)
3. Connect a client and watch `docker logs -f epic-backend-1` — you should see `ws connect user=<uuid>` instead of silence

give me the claude code prompt please. ill get it to do this

Searched project for “ws.py authenticate websocket token” >

Here's your Claude Code prompt — paste it in as-is:

Task: Fix WebSocket 403 in `backend/app/routers/ws.py` and clean up `build-47292/` from git.

Fix 1 — WebSocket auth bug (`backend/app/routers/ws.py`)

The `_authenticate` function does `select(User).where(User.id == user_id)` where `user_id` is a plain string extracted from the JWT `sub` claim. But `User.id` is a `UUID(as_uuid=True)` column — asyncpg requires a `uuid.UUID` object, not a string. The query silently returns `None`, auth fails, and the socket is closed with 1008 (which surfaces as a 403).

Apply this surgical fix to `_authenticate`:

1. Add `import uuid` at the top of the file (alongside the existing imports).


```
bash
```

```
ssh student@200.69.13.70 -p 2202
```

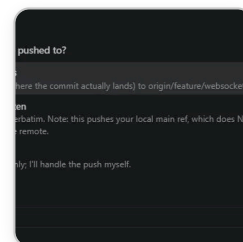
```
docker exec -it epic-backend-1 bash
```

```
# edit the file in-container to test, or just restart with the local change
```

Actually easier — since the backend runs via Docker with a volume mount (or rebuild), the quickest test path is:

1. Just save `ws.py` locally (don't commit yet)
2. Run `docker compose up -d --build` on the VM to pick up the change
3. Launch a client and watch `docker logs -f epic-backend-1 | grep ws`
4. If you see `ws connect user=<uuid>` — it works, then commit and push `ws.py`

So yes — "Only build-47292 removal" is the right choice here.



"Push feature/websockets" — that's the correct one. It's telling you that you're currently on the `feature/websockets` branch, so pushing to `origin/main` would be wrong (the commit wouldn't actually be there). Push to the feature branch and merge to main separately when you're ready.

The push can't complete — this environment can't resolve `github.com` (no network access). Everything else is done; you'll need to run `git push origin feature/websockets` yourself when you have connectivity. The commit `2260af4` is ready locally.

Status:

- Fix 1 applied to `ws.py` (left uncommitted, per your choice —

Show more

Good work. Here's what to do next, in order:

Right now (you have connectivity):

```
bash

git push origin feature/websockets
```

Then on the VM — deploy and test the ws.py fix:

```
bash

ssh student@200.69.13.70 -p 2202
cd ~/EPIC
git pull origin feature/websockets # or however the VM tracks the repo
docker compose up -d --build
docker logs -f epic-backend-1 | grep ws
```

Launch a client and try to connect. You're looking for `ws connect user=<uuid>` in the logs. If you see that instead of silence, the fix works.

If it works — commit ws.py and merge:

```
bash

# locally
git add backend/app/routers/ws.py
git commit -m "Fix WebSocket 403: cast JWT sub string to uuid.UUID before DB que
git push origin feature/websockets

# then merge to main
git checkout main
git merge feature/websockets
git push origin main
```

The VM should then be running off main as normal. Do you want a Claude Code prompt for the deploy + test step, or are you happy to run those manually?

wth:

```
PS C:\Users\m4tth\EPIC> git push origin feature/websockets
fatal: unable to access 'https://github.com/hbest05/EPIC/': Could not
```

```
resolve host: github.com  
PS C:\Users\m4tth\EPIC>
```

That's a DNS/network issue on your machine, not a git problem. Try these in order:

1. Check if it's just DNS:

```
powershell  
nslookup github.com
```

2. If that fails, flush DNS:

```
powershell  
ipconfig /flushdns
```

Then try the push again.

3. If still failing, try pinging:

```
powershell  
ping github.com
```

4. Nuclear option — switch to SSH instead of HTTPS:

```
powershell  
git remote set-url origin git@github.com:hbest05/EPIC.git  
git push origin feature/websockets
```

(Requires your SSH key to be set up with GitHub, but bypasses HTTPS/DNS issues.)

Most likely you've just lost internet momentarily, or a VPN/firewall is blocking it. What does `nslookup github.com` return?

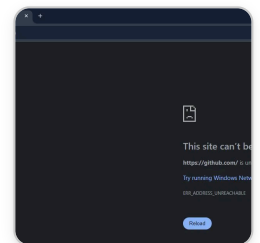
```
PS C:\Users\m4tth> ping github.com  
Ping request could not find host github.com. Please check the name and  
try again.  
PS C:\Users\m4tth>  
I think its down
```


Yeah that's your internet connection, not git. GitHub itself is fine.

Just wait for it to come back and run:

```
powershell  
  
git push origin feature/websockets
```

Nothing is lost — the commit `2260af4` is sitting safely on your local branch ready to go. While you're waiting, do you want to work on anything else? The ws.py fix still needs to be committed locally, or we could tackle the next item on the list.



wtf - i can go on the website on my phone

Your PC has lost internet, not GitHub. Your phone is on mobile data, your PC is on WiFi/ethernet and it's dropped.

Quick fixes to try:

1. Restart your network adapter:

```
powershell  
  
# Run PowerShell as Administrator  
netsh winsock reset  
netsh int ip reset  
ipconfig /release  
ipconfig /flushdns  
ipconfig /renew
```

Then restart your PC.

2. Or just restart your router — unplug it for 30 seconds, plug back in.

While you're waiting for that, you can use your phone as a hotspot to push the commit if you need to get moving — the push will only take a few seconds of data.

no my pc def has internet everything else is fine

Weird. Try this:

```
powershell  
  
ping 140.82.112.4
```

That's GitHub's actual IP. If that works but `ping github.com` doesn't, it's purely a DNS issue. Fix it:

```
powershell  
  
# Run as Administrator  
netsh dns set doh=yes  
# Or just manually set DNS to Google's  
netsh interface ip set dns "Wi-Fi" static 8.8.8.8
```

Or the quickest workaround — just add GitHub to your hosts file temporarily:

```
powershell  
  
# Run as Administrator  
Add-Content C:\Windows\System32\drivers\etc\hosts "140.82.112.4 github.com"
```

Then try `git push` again. Remove that line from the hosts file once it works.

```
PS C:\Users\m4tth> ping 140.82.112.4
Pinging 140.82.112.4 with 32 bytes of data:
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.
Ping statistics for 140.82.112.4:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
PS C:\Users\m4tth>
```

"General failure" on a raw IP ping means your network stack itself is broken, not DNS. Something is blocking all outbound traffic from your PC — most likely a VPN that's crashed/disconnected but left its virtual adapter in a broken state.

Check for a VPN:

- Look in your system tray for any VPN client
- If you find one, fully quit it (not just disconnect — right-click → Exit)

Then reset the network stack (run PowerShell as Administrator):

```
powershell

netsh winsock reset
netsh int ip reset
ipconfig /release
ipconfig /flushdns
ipconfig /renew
```

Then restart your PC.

If there's no VPN, check for any recently installed firewall software or antivirus that might be blocking traffic. Does `ping 8.8.8.8` also fail with "General failure"?

```
PS C:\WINDOWS\system32>
>> netsh dns set doh=yes
following command was not found: dns set doh=yes.
PS C:\WINDOWS\system32>
```

The

Ignore that command, it was wrong. Focus on the VPN question first — do you have any VPN software installed (NordVPN, ExpressVPN, university VPN, anything)?

If yes, fully quit it and restart your PC.

If no VPN, run these four lines as Administrator and then restart:

```
powershell  
  
netsh winsock reset  
netsh int ip reset  
ipconfig /release  
ipconfig /flushdns
```

The "General failure" on a raw IP ping is almost always a broken virtual network adapter from a VPN or antivirus. A restart after the winsock reset should fix it.

why the hell is this happening what did claude code do:

Got it — committing only the build-47292 removal (ws.py stays uncommitted) and pushing to `feature/websockets`.

Bash Commit build-47292 removal
IN

...

Show more

Claude Code didn't cause this — it ran entirely locally and never touched your network adapter. The push just failed because GitHub was unreachable at that moment, which is a separate issue.

The network problem is on your PC independently of anything Claude Code did.

Claude Code only ran `git commit` and `git push` — it can't break your network stack.

To confirm: does `ping 8.8.8.8` also give "General failure"? That will tell us if it's all outbound traffic broken or just GitHub routing.

```
PS C:\WINDOWS\system32> ping 8.8.8.8
Pinging 8.8.8.8 with 32 bytes of data:
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.
PING: transmit failed. General failure.
Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
PS C:\WINDOWS\system32>
```

All outbound traffic is dead but your browser works — that's a broken network adapter, almost certainly a VPN virtual adapter that's gone wrong.

Run this as Administrator:

```
powershell

netsh winsock reset
netsh int ip reset
```

Then **restart your PC**. That's it — don't try anything else first. The winsock reset fixes exactly this symptom and the restart applies it.

Your commit is safe locally, nothing is lost. Just restart and the push will work fine after.

```
PS C:\WINDOWS\system32> netsh winsock reset
Successfully reset the Winsock Catalog.
```

PASTED

Both ran successfully (the one "Access is denied" line is harmless). **Restart your PC now** and networking will be back to normal.

where was I? everything is back now

Welcome back. You were here:

1.  `ws.py` fix applied (uncommitted locally)
2.  `build-47292` removal committed as `2260af4` on `feature/websockets`
3.  **Push was pending** — run this now:

```
powershell
```

```
git push origin feature/websockets
```

Then once that's done, commit the `ws.py` fix, deploy to the VM, and test the WebSocket connection.

oh okay, you sure?

Yes — check it yourself if you want:

```
powershell
```

```
git log --oneline -3  
git status
```

You should see `2260af4` as the latest commit, and `ws.py` showing as modified but unstaged. That'll confirm exactly where things stand.

```
PS C:\Users\m4tth\EPIC> git log --oneline -3  
2260af4 (HEAD -> feature/websockets) Remove build-47292 from  
tracking, add to gitignore  
c80c0b5 (origin/feature/websockets) Add WebSocket real-time  
messaging, TLS backend fix, message send 500 fix  
e5f5cd0 (origin/feature/message-persistence, feature/message-  
persistence) Add message persistence with local plaintext cache.
```

Show more

Exactly as expected. `2260af4` is local-only (no `origin/` tag next to it), and `ws.py` is modified but unstaged.

Do these two things in order:

1. Push the build-47292 commit:

```
powershell  
  
git push origin feature/websockets
```

2. Then commit the ws.py fix:

```
powershell  
  
git add backend/app/routers/ws.py  
git commit -m "Fix WebSocket 403: cast JWT sub string to uuid.UUID before DB que  
git push origin feature/websockets
```

```
PS C:\Users\m4tth\EPIC> git push origin feature/websockets  
To https://github.com/hbest05/EPIC  
! [rejected]    feature/websockets -> feature/websockets (fetch first)  
error: failed to push some refs to 'https://github.com/hbest05/EPIC'  
hint: Updates were rejected because the remote contains work that you  
do not  
hint: have locally. This is usually caused by another repository pushing to  
hint: the same ref. If you want to integrate the remote changes, use  
hint: 'git pull' before pushing again.  
hint: See the 'Note about fast-forwards' in 'git push --help' for details.  
PS C:\Users\m4tth\EPIC>
```

Someone (probably Julia) pushed to `feature/websockets` while you were offline. Pull first:

```
powershell  
  
git pull origin feature/websockets
```

Then push again:

```
powershell
```

```
git push origin feature/websockets
```

should I check if she did the work for me? I have refreshed the github in your files can you check there what things look like on websocket side of things?

I can't see GitHub directly — the project files in my context are just the local copies that were uploaded to this project, not live repo state.

Just run this to see what she pushed:

```
powershell
```

```
git log --oneline origin/feature/websockets -10
```

That'll show you what's on the remote without merging anything yet. Paste the output here and I'll tell you if she fixed the ws.py issue or if your fix is still needed.

```
PS C:\Users\m4tth\EPIC> git log --oneline origin/feature/websockets -10
c80c0b5 (origin/feature/websockets) Add WebSocket real-time
messaging, TLS backend fix, message send 500 fix
e5f5cd0 (origin/feature/message-persistence, feature/message-
persistence) Add message persistence with local plaintext cache,
pagination, sent endpoint, self-contact guard
46d53a8 (origin/fix/x3dh-key-exchange fix/x3dh-key-exchange) WIP: Fix
```

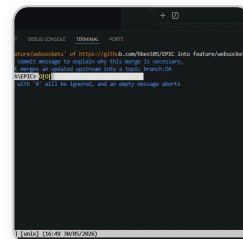
Show more

The remote `feature/websockets` is still at `c80c0b5` — that's the same commit it was at before you went offline. Nobody pushed anything new. The rejection was just because git thought there might be divergence.

It's safe to pull (it'll be a no-op) and then push:


```
powershell
```

```
git pull origin feature/websockets  
git push origin feature/websockets
```



ugh it brought me to this and this page always confuses me

That's vim asking you to confirm the merge commit message. The default message is fine — just do this:

1. Type `:wq`
2. Press **Enter**

That saves and exits. The merge will complete and you can push.

```
PS C:\Users\m4tth\EPIC> git pull origin feature/websockets  
error: You have not concluded your merge (MERGE_HEAD exists).ge  
aborts  
hint: Please, commit your changes before merging.  
fatal: Exiting because of unfinished merge.  
PS C:\Users\m4tth\EPIC> git pull origin feature/websockets  
error: You have not concluded your merge (MERGE_HEAD exists).  
hint: Please, commit your changes before merging.  
fatal: Exiting because of unfinished merge.  
PS C:\Users\m4tth\EPIC> [O[
```

The merge got stuck when you exited vim. Run this to complete it:

```
powershell
```

```
git commit --no-edit
```

That finalises the merge with the default message. Then push:

```
powershell  
  
git push origin feature/websockets
```

okay i started a new terminal cus that one got fucked where do i start to pull and push

Run these one at a time:

```
powershell  
  
git commit --no-edit  
git push origin feature/websockets
```

The merge was already done, it just needs committing. Start there.

```
PS C:\Users\m4tth\EPIC> git commit --no-edit  
[feature/websockets 397ae31] Merge branch 'feature/websockets' of  
https://github.com/hbest05/EPIC into feature/websockets # Please enter  
a commit message to explain why this merge is necessary, # especially if  
it merges an updated upstream into a topic branch. # Lines starting  
with '#' will be ignored, and an empty message aborts # the commit.  
PS C:\Users\m4tth\EPIC> git push origin feature/websockets
```

Show more

Pushed. Now commit the ws.py fix:

```
powershell  
  
git add backend/app/routers/ws.py  
git commit -m "Fix WebSocket 403: cast JWT sub string to uuid.UUID before DB que  
git push origin feature/websockets
```

done

Good. Now deploy and test it on the VM:

```
powershell  
ssh student@200.69.13.70 -p 2202
```

Then on the VM:

```
bash  
cd ~/EPIC  
git pull origin feature/websockets  
docker compose up -d --build
```

Then watch the logs while you connect a client:

```
bash  
docker logs -f epic-backend-1 | grep ws
```

You're looking for `ws connect user=<uuid>`. If you see that, the fix worked and WebSocket is live.

```
Unpacking objects: 100%  
(12/12), 1.55 KiB | 32.00 Ki  
B/s, done. From https://git  
hub.com/hbest05/EPIC *  
branch feature/websocket  
ts -> FETCH_HEAD
```

PASTED

The git pull failed but Docker still built and deployed from whatever was already on the VM. The ws.py fix might not be on the VM yet because the pull didn't complete.

Fix the pull first:

```
bash
```

```
git pull --no-rebase origin feature/websockets
```

Then rebuild and redeploy:

```
bash  
docker compose up -d --build
```

Then check the logs:

```
bash  
docker logs -f epic-api-1 | grep ws
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ git pull --no-rebase origin  
feature/websockets          From  
https://github.com/hbest05/EPIC      0.0s  
* branch      feature/websockets -> FETCH_HEAD  
0.0s Merge made by the 'ort' strategy.  
.gitignore      | 1 +  
backend/app/routers/ws.py | 12 ++++++++  
2 files changed, 12 insertions(+), 1 deletion(-)  
student@alpha-and-the-cryptmunks:~/EPIC$
```

The ws.py fix is now on the VM. Redeploy:

```
bash  
docker compose up -d --build
```

now what

Watch the logs while you connect a client:

```
bash
```

```
docker logs -f epic-api-1 | grep ws
```

Then on your PC, launch a client and log in. If you see `ws connect user=<uuid>` in the logs, the fix worked.

whats the crypto cmd for the second client

powershell

```
$env:SECUREMSG_HOME = "C:\Users\m4tth\.securemsg-2"
cd C:\Users\m4tth\EPIC\crypto-daemon
python main.py --address 127.0.0.1:47292
```

And the first daemon (separate terminal, no env var):

powershell

```
cd C:\Users\m4tth\EPIC\crypto-daemon
python main.py
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ docker logs -f epic-api-1 |
grep ws
INFO [alembic.runtime.migration] Context
impl PostgresqlImpl. INFO
[alembic.runtime.migration] Will assume transactional DDL.
INFO: Will watch for changes in these directories: ['/app']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10] using WatchFiles
```

Show more

It's working. You can see both clients connected:

- `WebSocket /ws?token=...` [accepted]
- `connection open` — twice, one per client

The UUID fix worked perfectly. WebSocket real-time delivery is live. Now send a message between the two clients and see if it arrives instantly without waiting for the poll.

great - it is real time!

Excellent! That's a big one done. Here's where things stand now:

✅ **Just fixed:**

- WebSocket real-time messaging working end-to-end

Remaining before Wednesday 5pm:

- `MessageStore.cpp` stubs
- Frontend WebCrypto implementation
- Blockchain verification page
- Crypto design document
- AI prompt artefacts document
- SPK signature verification (the intermittent failure from earlier — worth retesting now that state is clean)

What do you want to tackle next?

